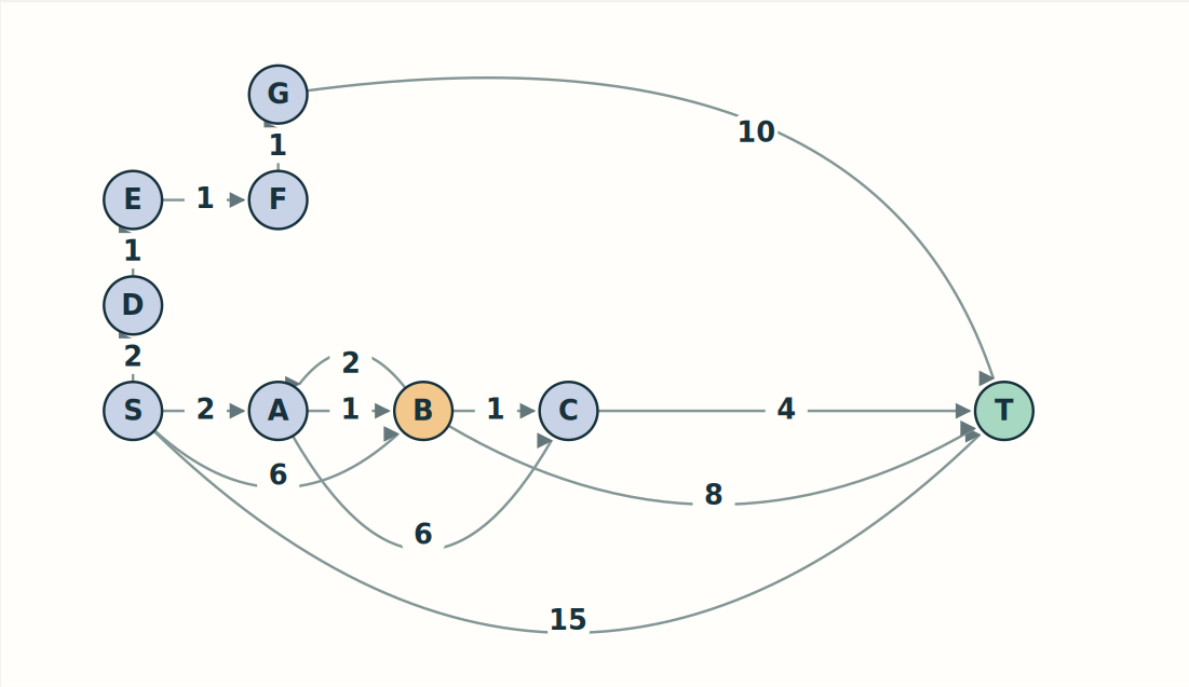


Why may this record be discarded?

Main graph · Actual Python state after the highlighted operation



The queue record is obsolete, not the node. B was improved from 6 to 3. Do not scan its outgoing edges again.

Node	g	Parent
S	0	—
A	2	S
B	3	A
C	4	B
D	2	S
E	3	D
F	4	E
G	5	F
T	8	C

Frontier in priority order (not heap storage)

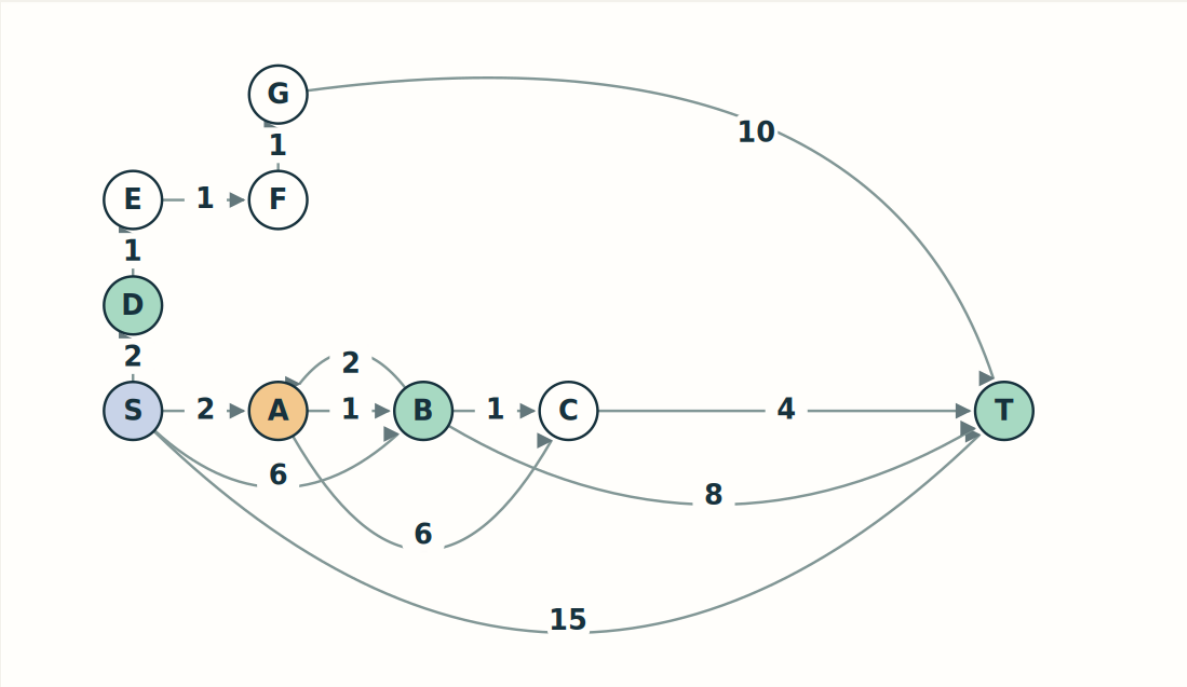
Node	Priority	Queued g
C ×	8	8
T	8	8
T ×	11	11
T ×	15	15

Expansions: 8 Stale skips: 1 Re-expansions: 0 Peak queue: 6

```
algorithms/dijkstra.py / function-relative line 16 (just executed)
stats.stale_pops += 1
```

Can we repair the entry instead?

Main graph · Actual Python state after the highlighted operation



One live record per frontier node. A strict decrease changes its priority and the node-to-index map.

Node	g	Parent
S	0	—
A	2	S
B	3	A
C	∞	—
D	2	S
E	∞	—
F	∞	—
G	∞	—
T	15	S

Frontier in priority order (not heap storage)

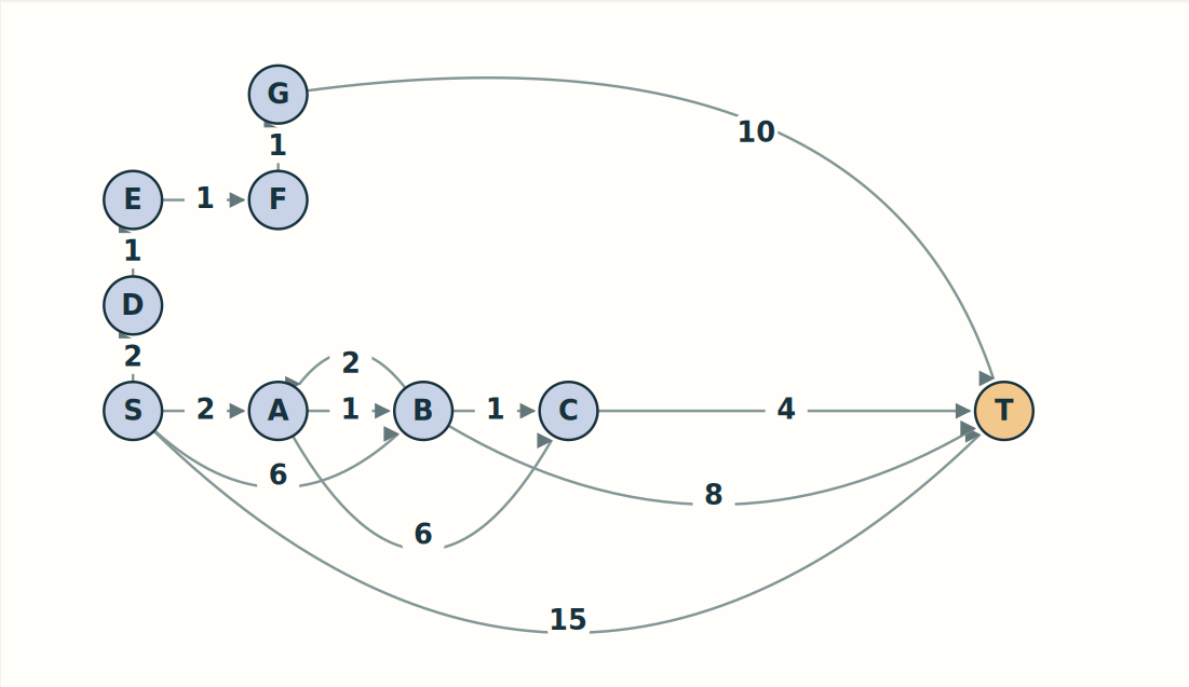
Node	Priority	Queued g
D	2	2
B	3	3
T	15	15

Expansions: 2 Stale skips: 0 Re-expansions: 0 Peak queue: 4

```
algorithms/dijkstra.py / function-relative line 26 (just executed)
queue.decrease_key(v, candidate)
```

When can this route be certified?

Main graph · Actual Python state after the highlighted operation



The current goal entry was extracted after the stale check. Cost 8 is certified; remaining queue records need not be processed.

Node	g	Parent
S	0	—
A	2	S
B	3	A
C	4	B
D	2	S
E	3	D
F	4	E
G	5	F
T	8	C

Frontier in priority order (not heap storage)

Node	Priority	Queued g
T ×	11	11
T ×	15	15

Expansions: 8 Stale skips: 2 Re-expansions: 0 Peak queue: 6

algorithms/dijkstra.py / function-relative line 20 (just executed)

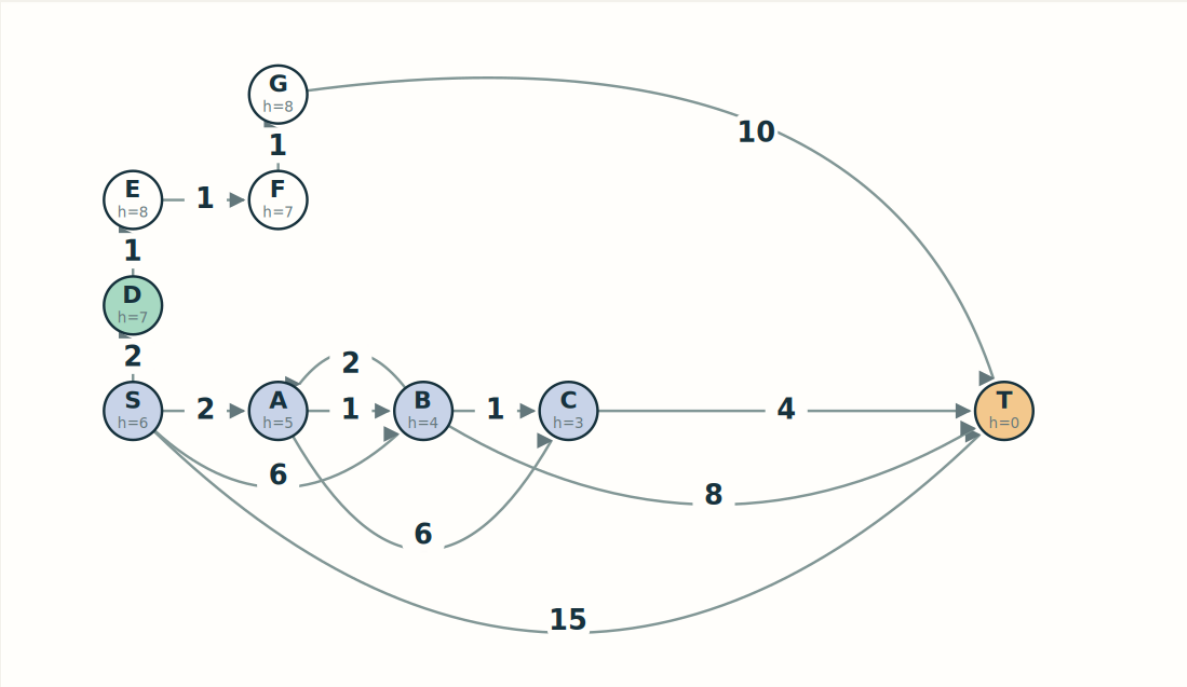
```
return finish(dist, parent, start, target, stats, finalized)
```

Gold = current Green = frontier Blue = finalized (Dijkstra) / expanded (A*, may reopen)

Static snapshots generated from the Python traces; animation playback speed is not runtime.

Which information changed the order?

Main graph · Actual Python state after the highlighted operation



With a consistent h, A* reaches the same cost 8 after four expansions. D remains a frontier candidate; fewer expansions is not a universal runtime guarantee.

Node	g	h	Parent
S	0	6	—
A	2	5	S
B	3	4	A
C	4	3	B
D	2	7	S
E	∞	8	—
F	∞	7	—
G	∞	8	—
T	8	0	C

Frontier in priority order (not heap storage)

Node	Priority	Queued g
D	9	2
B ×	10	6
C ×	11	8
T ×	11	11
T ×	15	15

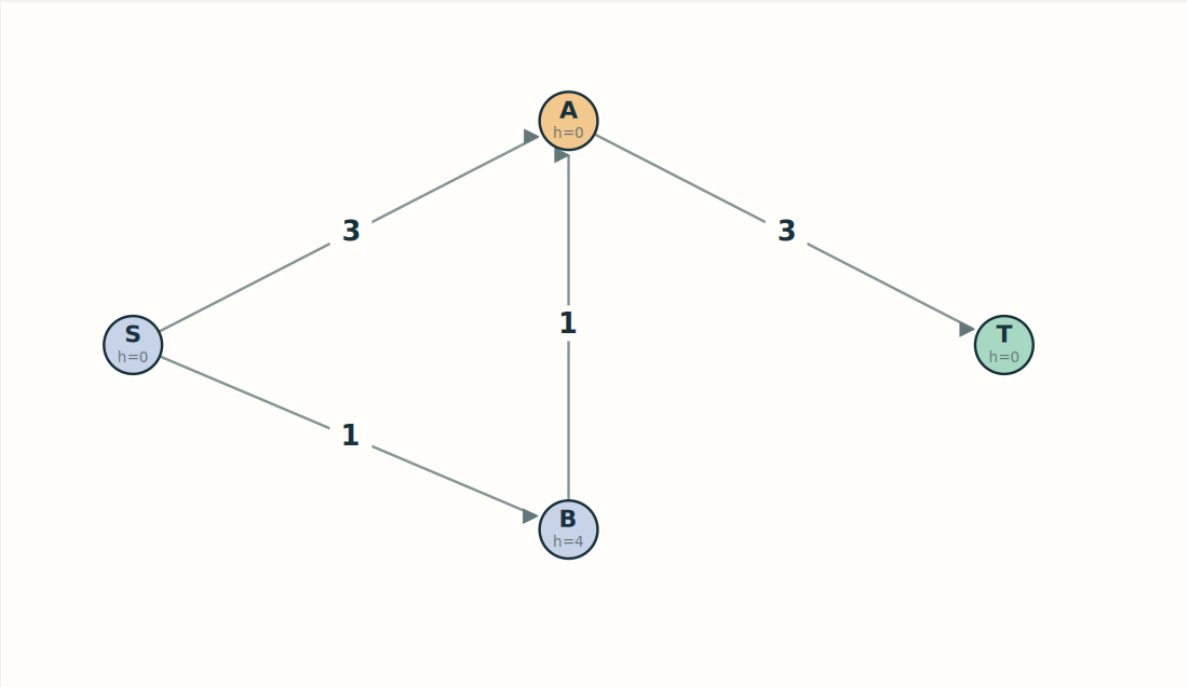
Expansions: 4 Stale skips: 0 Re-expansions: 0 Peak queue: 6

```
algorithms/astar.py / function-relative line 20 (just executed)
return finish(dist, parent, start, target, stats, {target})
```

Gold = current Green = frontier Blue = finalized (Dijkstra) / expanded (A*, may reopen)
Static snapshots generated from the Python traces; animation playback speed is not runtime.

Should A be ignored or processed again?

Admissible but inconsistent · Actual Python state after the highlighted operation



A was expanded at g=3 and is now reached at g=2. Reopening propagates that improvement. The correct final cost will be 5.

Node	g	h	Parent
S	0	0	—
A	2	0	B
B	1	4	S
T	6	0	A

Frontier in priority order (not heap storage)

Node	Priority	Queued g
T	6	6

Expansions: 3 Stale skips: 0 Re-expansions: 1 Peak queue: 2

algorithms/astar.py / function-relative line 21 (just executed)

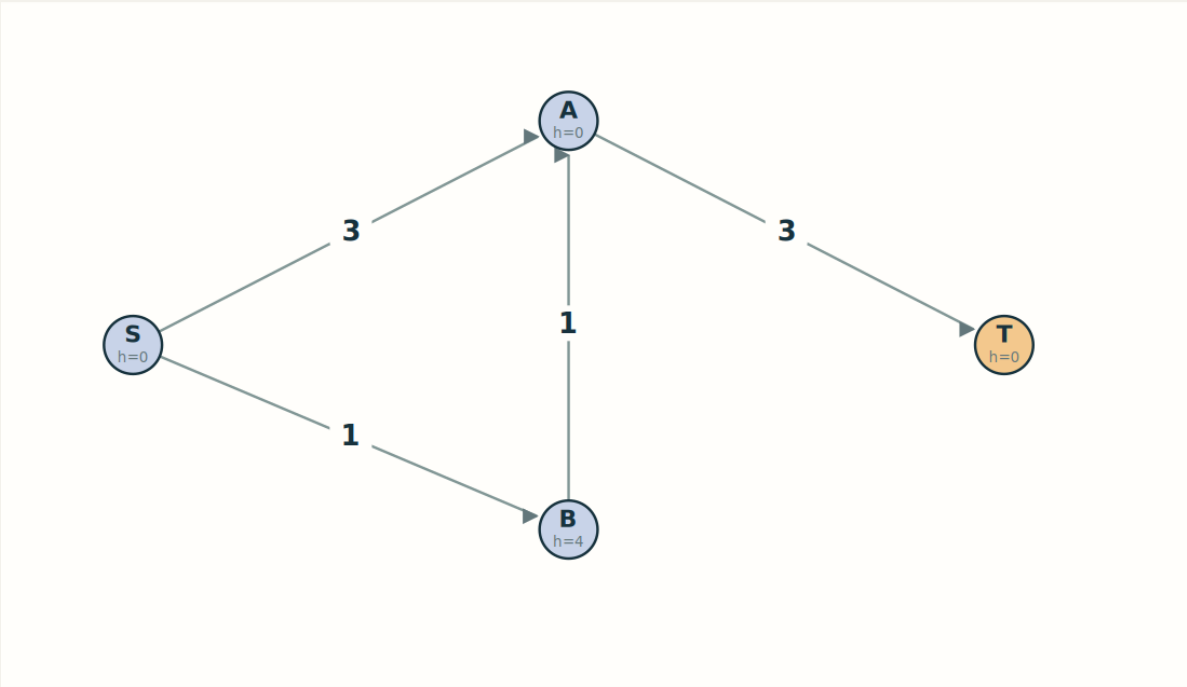
```
stats.reexpansions += int(u in expanded)
```

Gold = current Green = frontier Blue = finalized (Dijkstra) / expanded (A*, may reopen)

Static snapshots generated from the Python traces; animation playback speed is not runtime.

Which assumption did permanent closure require?

Admissible but inconsistent · Actual Python state after the highlighted operation



Node	g	h	Parent
S	0	0	—
A	3	0	S
B	1	4	S
T	6	0	A

Frontier in priority order (not heap storage)

Node	Priority	Queued g
empty	—	—

DELIBERATELY WRONG: admissibility alone did not justify closing forever. This variant returns 6; reopening returns the optimum 5.

Expansions: 3 Stale skips: 0 Re-expansions: 0 Peak queue: 2

```
algorithms/experiments.py / function-relative line 19 (just executed)
return finish(dist, parent, start, target, stats, set())
```